Revision 0:
Validation and Verification

Group 8 - C.A.V.E: Cave Assessment and Visualization Equipment

Abdul Rahim Khan
Andrew Brink
Gokberk Yilmaz
Nicholas Trimble
Tabish Faisal

February 1, 2026

Table I: Revision History

Date	**Developer(s)**	**Change**
February 1, 2026	All Members	Initial Draft

Glossary

Term	Definition
IMU	Inertial measurement unit, providing acceleration and rotational data
LiDAR	Light Detection and Ranging: a sensor that uses a rotating laser to measure distances to objects.
Point cloud	A collection of coordinate points in 3D space
Portable device	The portion of the design that is carried with the user/operator for mapping an enclosed space
TOF sensor	Time of Flight sensor, which uses pulses of infrared light and a sensor that measures the time at which the light is received back to create a point cloud.
ICP	Iterative closest point algorithm minimizes the differences between subsequent point clouds to reconstruct 3D surfaces.

Table II: Terms and definitions

Contents

1 Introduction

1.1 Purpose

The purpose of this document is to validate and verify that the project requirements are feasible and achieve the objectives laid out in the contract. This serves as a checkpoint in the development process where we assess whether our software and hardware are capable of fulfilling the project requirements. The verificiation process provides a set pass/fail tests in which the status of the current revision is tested against ensuring requirement completion.

1.2 Scope

This validation and verification encompass the entire C.A.V.E. design, including software, electrical, and mechanical components. Requirements are defined in Revision 0: Requirements and Hazard Analysis. The scope covers all the behavioural, timing and software/hardware requirements tied the system. Additionally, this document addresses the modifications to the original requirements based on the implementation experience and feasibility checks conducted during the development process.

2 Validation

The validation process examines each requirement through a qualitative metric, assessing whether our system design choices fulfill and support the overall project objectives.

2.1 Behavioural Requirements

2.1.1 RB1

> The device will produce a point cloud C_point_cloud representing the physical surfaces mapped by the sensors (M_tof_data, M_imu_gyro) while recording is enabled (M_start_stop == true).

The C.A.V.E system achieves this fundamental requirement by fusing the data from multiple sensors: the ToF sensors actively (M_start_stop == true) provides the point cloud (M_tof_data) at each consecutive time-stamps while the IMU outputs orientation and rotational data (M_imu_gyro) that allows the system to properly map out successive point cloud in a 3D global space.

The oriented point clouds are now passed through our ICP algorithm which further refines the reconstruction of the scanned environment by aligning the consecutive point cloud frames, minimizing the distance between each capture and producing a coherent and recognizable final point cloud (C_point_cloud).

2.1.2 RB2 (Removed)

> The confidence in the accuracy of C_point_cloud shall be represented by C_confidence

Our second behaviour requirement was as follows: "The confidence in the accuracy of C_point_cloud shall be represented by C_confidence". In the process of designing and implementing the algorithms for fusing the sensor data, we have concluded that this requirement is not necessary for the fulfillment of our goal. Providing a metric for the confidence of the system has proved to be a difficult thing and is not required to achieve the primary goal, which is to produce the point cloud. Our

implementation either succeeds in fusing the data, or it fails to do so because of an error, but there is no intermediate state where the model is 50% correct.

We considered replacing this requirement with a specification that the final point cloud needs to be accurate to the actual cave within a certain threshold, but there are a couple of issues with this. This would be practically the same as the first requirement. However, it would additionally include error threshold numbers which we would have to arbitrarily decide. Testing this requirement would be impossible without additional expensive equipment like 3D LiDAR. Instead of adding this type or requirement, we decided that our testing for accuracy will have to be qualitative rather than quantitative, and that checking that the model is easily recognizable will be our most important test. This conclusion was based on discussions with Dr. Wassyng.

2.1.3 RB3

> C_record_status toggles between indicating recording and standby when M_start_stop is toggled by the user. If the system encounters an error it cannot recover from, it must display an error status on C_record_status.

Completion of this requirement happens via the RPi Head 'hat' board on the Raspberry Pi. System software reacts to the pressing of Button 1 to transition from standby to recording. The LED U10 indicates that the system is actively recording the environment, and once Button 1 is pressed again this LED deactivates. Communicating and error on C_record_status is achieved by illuminating LED1 "Sensing Error". This requirement remains the same as the original specification, though the design of the buttons could be difficult to reach or differentiate when the Pi is mounted at the back of the helmet. Additionally, the LEDs may be difficult see by the person wearing the device, however this can be mitigated by operating with another person as a duo, a standard caving safety practice.

2.1.4 RB4

> C_record_time will display remaining available recording time by taking the minimum of M_SOC/const_power_rate and M_storage_left/const_datarate.

The Raspberry Pi is powered through a portable powerbank which has a seven-segment display showing the remaining battery percentage. This can be used to calculate the time remaining (C_record_time) for the data recording process.

2.1.5 RB5 (Modified)

> Updated: The quality of the point cloud produced by the system must be independent of all external light.

Our 5th behavioural requirement previously was that the confidence level had to remain above a certain value when the system was operating in little to no light conditions. However, since the confidence level is no longer being used (see the discussion for RB2), this requirement needs to be altered. The reason for this requirement originally was to ensure that the system would operate properly in the dark conditions of a cave. In keeping with the original intent, we have modified this requirement to remove the mention of C_confidence.

This requirement means that all the sensors used should not be sensitive to the lack of natural light and should not be affected by any flashlights being used by the operator. The fulfilment of this requirement is possible because TOF sensors produce their own light, which is in the infrared range.

2.2 Timing

2.2.1 RT1

> Const_capt_rate shall be frequent enough to produce complete maps of an environment at a walking pace (3-5km/h)

—Insert explanation here—

2.2.2 RT2

> Const_capt_rate shall be low enough such that the system can complete a measurement of all sensor data before the next interval.

Rotating the point clouds with the IMU data produces coherent results. This sensor fusion results in a 3D model that is easy to parse with simple visualization software. We were able to run the ToF and IMU sensors simultaneously to map a small room accurately.

2.3 Hardware/Software Environment:

2.3.1 RE1

> The portable device shall not require an active internet connection to achieve the behavioural requirements RBX.

The device does not need to be connected to any form of internet or Bluetooth and is still able to run completely without hinderance. This can be shown by running the system without it connected to any form of internet or cellular device.

2.3.2 RE2

> The hardware should minimize encumbrance of the user to ensure exploration is not inhibited.

The entire device weighs less than two kilograms, as well as having a low profile, which mitigates any form of hinderance to the user it can cause.

3 Verification

In this section, the verification process establishes objective tests with a pass/fail criteria that determine whether each requirement has been successfully implemented in the current system revision. Images for the tests are provided in

3.1 Behavioural Requirements

Several of these tests are well defined, such as those related to the hardware, but some of the algorithm performance metrics are more difficult to define and as mentioned above the requirements may be updated. At the current state of the project, several failures are expected, as much of the device/interface integration remains to be completed.

Test	Procedure	Expected Result	Pass/Fail	Notes
T-RB1-0	Record data from ToF and IMU. Fuse data and pass oriented through ICP algorithm which outputs final point cloud.	Point cloud where the environment/scene can be recognized	Pass	Scan of small room was recognizable
T-RB2-0	-	-	-	Requirement removed
T-RB3-0	Press button 1 after system power on	U7, U10 illuminates; recording starts	Fail	U7 illuminates, U10 and recording pending software integration. Button press recognized.
T-RB3-1	Press button 2 after system power on	U8 illuminates	Pass	No software interaction yet. LED overly bright
T-RB4-0				
T-RB5-0	Use TOF sensor with the same scene in multiple lighting conditions (sunlight, darkness, flashlight)	All point clouds produced are identical	Pass (assumed)	I haven't actually done this test, unless you count when we tested the sensor in the Thode meeting room

Table 1: Verification Tests for Behavioural Requirements

3.2 Timing

Both IMU and ToF data come with timestamps. Using these timestamps, we can ensure both are within 100ms of each other. In our previous tests, we achieved a precision of around 50ms, which is more than enough for a walking pace.

To further improve this, we can switch to the Linux RT kernel which will make it easier for us to manage the real-time aspect of this project. The Linux RT API allows us to set deadlines for tasks, which would be essential if we need better precision.

3.3 Hardware / Software Environment

The tests are very empirical and easy to determine. To test whether or not the system can run without internet we will run the same program both connected to the internet, and not connected

to the internet, and it is expected that there will be no change in the result (as it is not internet dependent). The hardware environment is a bit harder to directly test due to variability, but the system is lightweight and has a small enough footprint it shouldn't hinder mobility. This could be tested by having the user traverse an area without the equipment, and with, and have weighted scaling to determine how much of a hinderance it is.

Test	**Procedure**	**Expected Result**	**Pass/Fail**	**Notes**
T-RE1-0	Capture the same arbitrary data with and without an internet connection	Both rosbags should have the same raw information, accounting for general noise	Pass	None
T-RE1-1	Run the ICP algorithm on the same initial data with and without an internet connection	The ICP should have similar results on both runs, accounting for variability in the algorithm	Pass	None
T-RE2-0	Measure the overall weight of the design	The entire system should be under 2 kg	Pass	Total Mass of all components is around 0.5-0.7 kg: Helmet, Raspberry Pi, Sensors
T-RE2-1	Traverse a fixed location at a fixed path with and without the equipment	It should feel the exact same with and without everything worn, or at least not hinder mobility by a good margin	-	This is difficult to empirically decide due to it being opinionated and different for other users, but a general gist is needed.

Table 2: Verification Tests for Hardware/Software Requirements

4 Appendix A: Verification Test Images

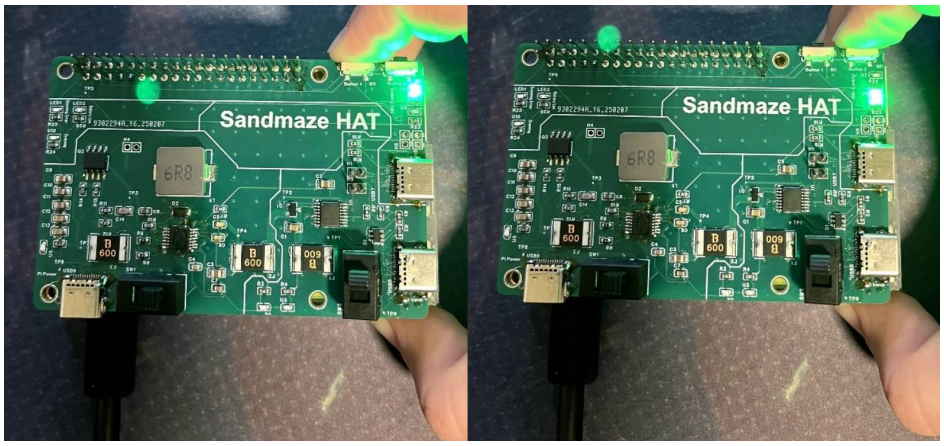

Figure 1: Test T-RB3-0/1

(a) Test T-RE2-0 (Helmet)

(b) Test T-RE2-0 (Raspberry Pi 4)

Figure 2: Test T-RE2-0